



Programação Orientada a Objetos

Prof. Rodrigo Colnago Contreras

Relatório Técnico: Desenvolvimento de Jogo Blackjack (21) em C#

Grupo 3:

Henrique Rodrigues Ribeiro - 176.515

Lael Kenzo Hayashi - 176.550

Matheus Dousseau -170455

Vinicius Elias – 176552

1. Introdução

Este documento detalha a implementação de uma aplicação desktop em C# utilizando Windows Forms (.NET). O projeto consiste numa simulação offline do clássico jogo de cartas Blackjack (também conhecido como 21). O desenvolvimento foi fortemente fundamentado no paradigma da Programação Orientada a Objetos (POO), garantindo um código modular, escalável e de fácil manutenção.

2. Descrição Lógica do Sistema

A arquitetura do sistema foi dividida entre a camada de lógica de negócios (as regras do jogo) e a camada de apresentação (a interface gráfica). A abstração do problema real para o código resultou na criação de diversas classes interligadas.

2.1. Arquitetura de Classes e POO

O sistema aplica os quatro pilares da POO: **Abstração** (modelagem das cartas e baralho), **Encapsulamento** (proteção de variáveis através de propriedades *getters/setters*), **Herança** (jogadores e banca partilham características base) e **Polimorfismo** (tratamento uniforme de subclasses).

- **Classe Carta:**
 - Responsável por representar uma carta individual.
 - **Atributos encapsulados:** valor (A, 2-10, J, Q, K), naipe (Paus, Copas, Espadas, Ouros), peso (valor numérico no jogo) e path (caminho da imagem associada).
 - **Lógica Interna:** Utiliza estruturas do tipo Dictionary com comparação *case-insensitive* (StringComparer.OrdinalIgnoreCase) para mapear automaticamente o valor textual da carta (ex: "K") para o seu peso no jogo (10) e para construir o caminho correto da imagem no disco local na inicialização.
- **Classe Baralho:**
 - Representa uma coleção de 52 objetos Carta. Demonstra o conceito de *Composição/Agregação*.
 - **Métodos principais:** * InicializarBaralho(): Utiliza laços de repetição aninhados (naipes x valores) para gerar as 52 cartas.
 - Embaralhar(): Implementa um algoritmo lógico de permutação aleatória utilizando a classe Random para misturar as posições dos elementos na lista.
 - ComprarCarta(): Remove e devolve a primeira carta do topo da lista (RemoveAt(0)), simulando a distribuição física de cartas.
- **Classe Abstrata JogadorBase:**
 - Classe fundacional que não pode ser instanciada diretamente. Contém os dados que qualquer participante de um jogo de cartas precisa ter.
 - **Atributos:** Nome e Mao (uma List<Carta> que guarda as cartas atuais).

- **Lógica do Ás (CalcularPontuacao):** O método itera sobre as cartas na mão, somando os pesos. Se a pontuação ultrapassar 21 e o jogador possuir um Ás (que inicialmente vale 11), o sistema subtrai 10 pontos do total para cada Ás disponível, convertendo dinamicamente o seu valor para 1, evitando que o jogador estoure de forma injusta.
- **Classes Jogador e Banca:**
 - Herdeiras de JogadorBase.
 - A classe Banca estende o comportamento padrão implementando a sua própria Inteligência Artificial através do método DeveComprarCarta(). Este método retorna true enquanto a pontuação da banca for menor que a do jogador e simultaneamente menor que 21.
- **Classe Jogo:**
 - Classe gestora. Agrupa e instancia o BaralhoAtual, o JogadorAtual e a BancaAtual.
 - O método VerificarVencedor() processa as regras de negócio finais: analisa se alguém estourou 21 e, na hora da paragem, compara os pontos. Se houver empate numérico, a lógica confere a vitória à Banca, conforme as regras estabelecidas.

2.2. Lógica de Interface e Assincronismo

A interface gráfica (Form1) atua como a ponte entre o utilizador e o motor do jogo. Para criar uma experiência imersiva sem bloquear a interface de utilizador (UI), o sistema utiliza **Programação Assíncrona** (async e await com Task.Delay).

- **Motor de Animação (DistribuirCartaAnimada):** Quando uma carta é solicitada, em vez de aparecer instantaneamente na posição final, a aplicação gera dinamicamente um objeto PictureBox transparente, atribui-lhe a imagem correspondente lida da pasta /deck_2/ e calcula o trajeto vetorial entre o baralho base e a mão do jogador. Um laço iterativo atualiza as coordenadas X e Y em pequenas frações de segundo, criando um efeito visual de deslizamento suave. O espaçamento dinâmico (espacamentoCartas) garante que as cartas se sobreponham visualmente, de forma semelhante ao mundo real.

3. Exemplos de Usos (Casos de Uso)

O sistema foi desenhado para ser intuitivo. Abaixo estão descritos os principais fluxos de interação e as respostas lógicas do sistema a cada ação do utilizador.

Exemplo 1: O Fluxo de Início e Vitória do Jogador

1. O utilizador clica em "Iniciar".
2. O sistema limpa a mesa virtual, instancia um novo Baralho, embaralha as cartas e distribui 4 cartas animadas e alternadas (2 para o Jogador viradas

para cima, 1 para a Banca virada para cima e 1 para a Banca virada para baixo).

3. A pontuação inicial do Jogador é 12. O utilizador clica em **"Pedir Carta"**.
4. O sistema processa o clique, retira uma carta do baralho e anima o seu deslocamento até à mão do jogador. A nova carta é um 8. A pontuação sobe para 20.
5. Satisfeito com o valor, o utilizador clica em **"Parar"**.
6. O sistema bloqueia os controlos, revela visualmente a carta oculta da Banca e inicia o turno autónomo da Banca.
7. A Banca, possuindo 15 pontos, detecta que tem menos pontos que o jogador. Pede uma carta. A carta é um 8. A Banca vai a 23 pontos e "estoura".
8. O sistema exibe uma caixa de diálogo: *"A Banca estourou! Você venceu."* e finaliza a partida.

Exemplo 2: Derrota Imediata (Estouro de 21)

1. Durante a partida, o jogador possui 15 pontos.
2. O utilizador decide arriscar e clica em **"Pedir Carta"**.
3. A carta desenhada e animada para a tela é um Rei (K), cujo peso numérico é 10.
4. O sistema invoca `CalcularPontuacao()`, detectando que a soma chegou a 25. O sistema valida que não existem ases na mão para reduzir o valor.
5. Instantaneamente, a interface bloqueia os botões "Pedir" e "Parar", revelando o ecrã de derrota: *"Você estourou 21! A Banca venceu."*

Exemplo 3: Vitória da Banca por Empate ou Maior Pontuação

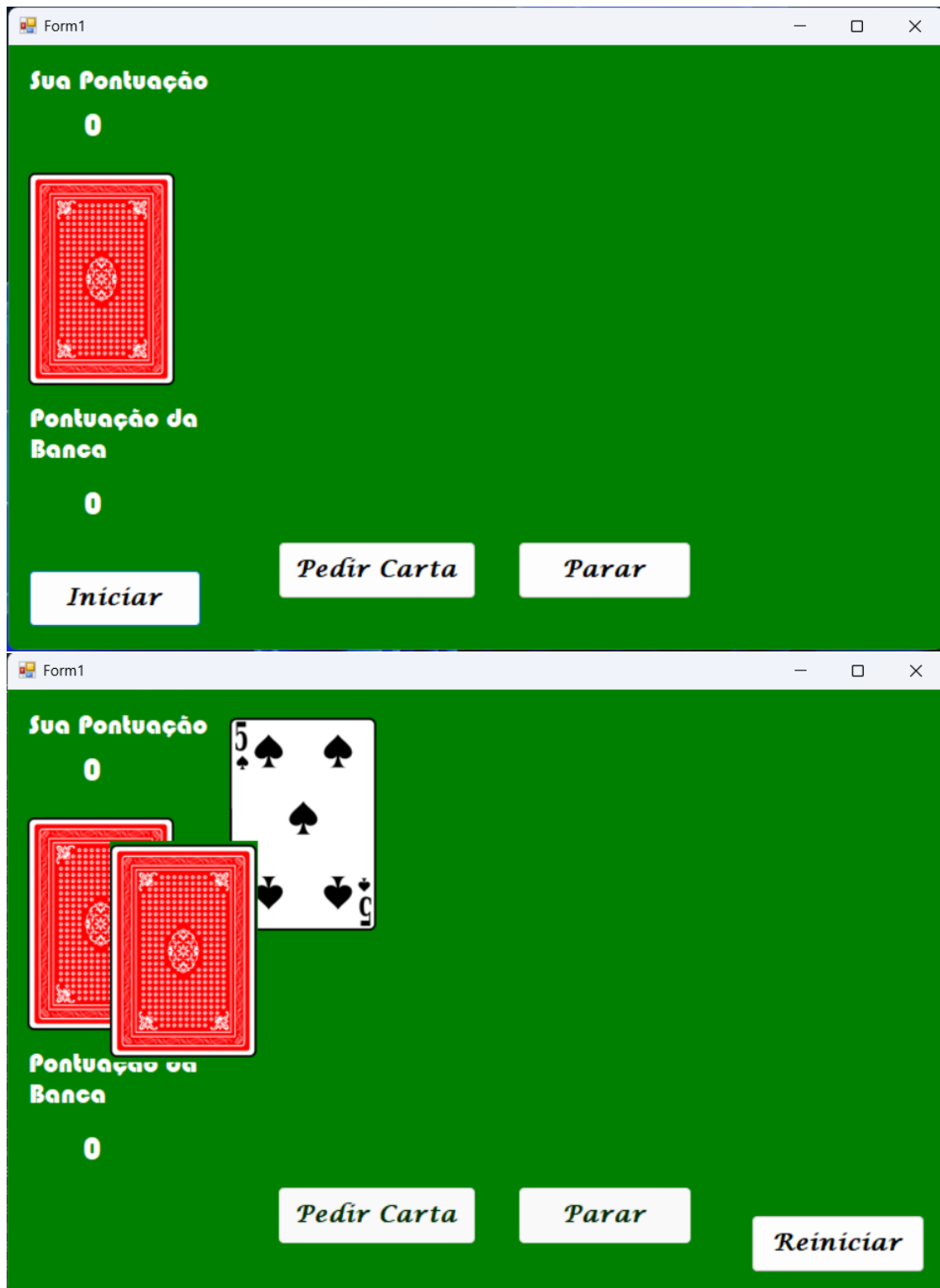
1. O jogador constrói uma mão sólida e atinge **19 pontos**, clicando de seguida em **"Parar"**.
2. A Banca revela a sua carta oculta e descobre que tem **13 pontos**.
3. Respeitando a lógica de `DeveComprarCarta()`, o sistema da Banca compra uma nova carta assincronamente (com pausa visual). A carta é um 6. A Banca passa a ter **19 pontos**.
4. A lógica de paragem é avaliada. Como a Banca já não tem menos pontos que o jogador (19 não é menor que 19), o laço de repetição cessa.
5. O método de verificação compara 19 do jogador contra 19 da banca. Pela regra de negócio (empate favorece a banca), o sistema encerra o jogo exibindo: *"A Banca venceu!"*

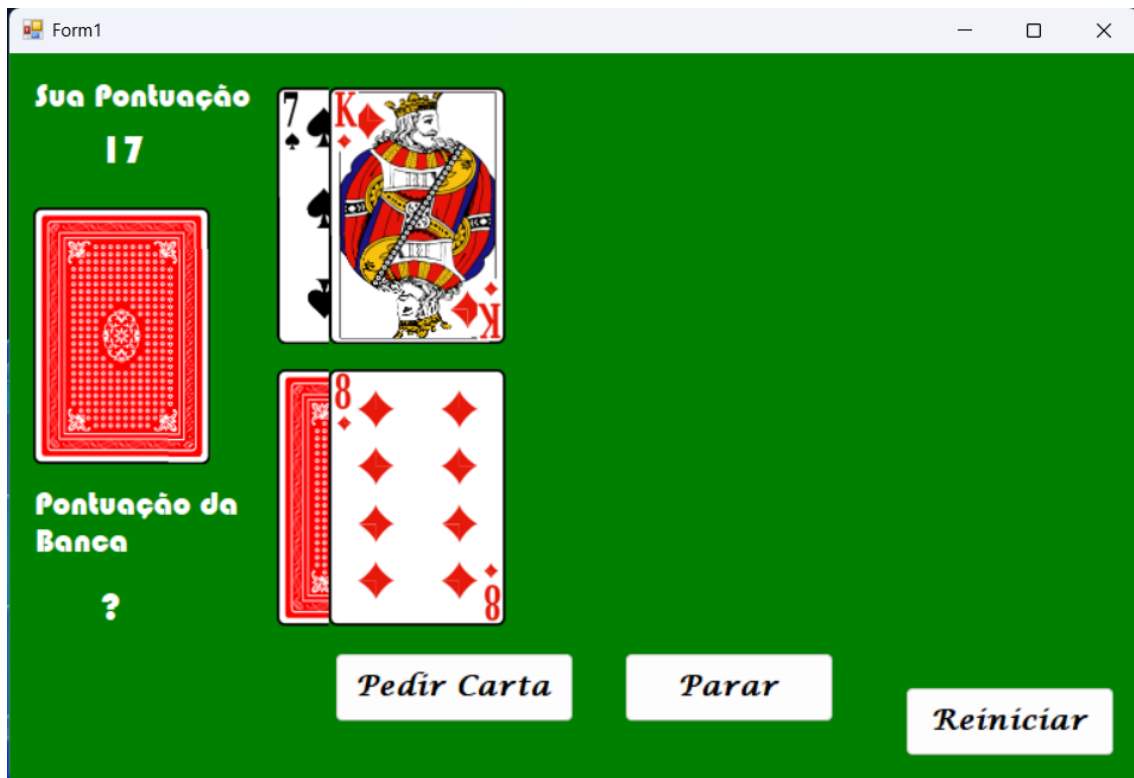
Exemplo 4: Utilização do Botão Reiniciar

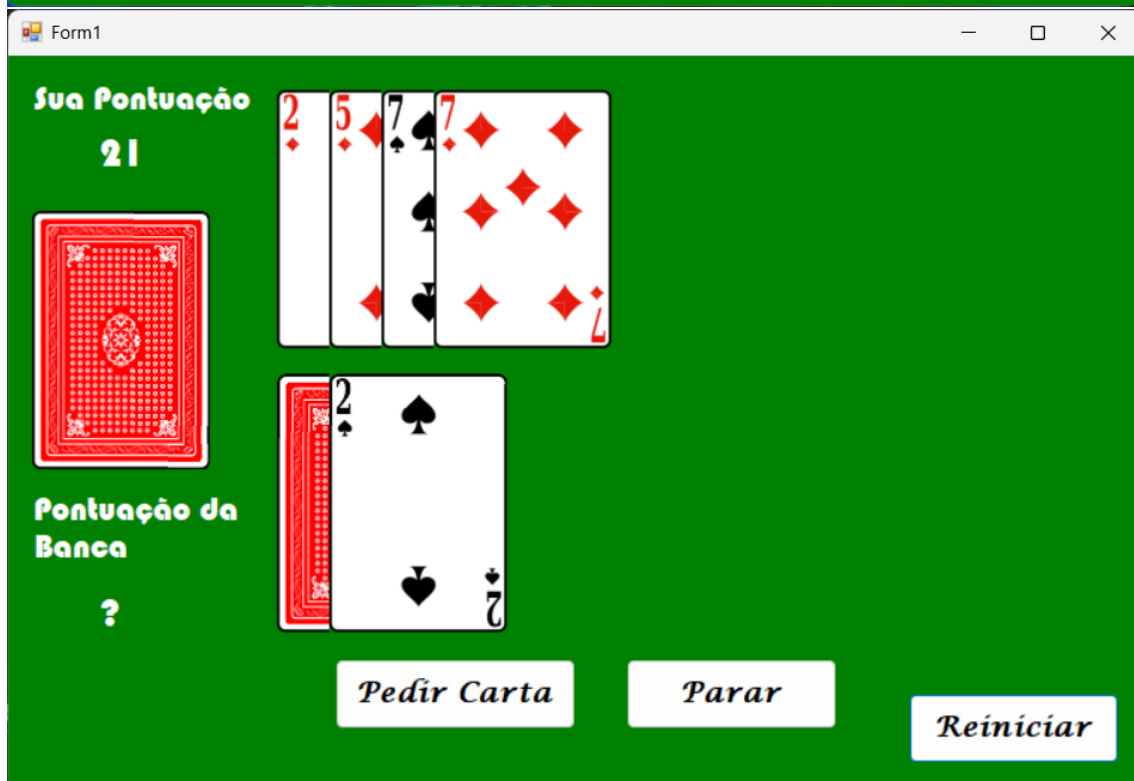
1. O jogo atual encontra-se finalizado, e a mesa está cheia de cartas exibidas.
2. O utilizador deseja uma nova partida e clica no botão **"Reiniciar"**.

3. O sistema reinicia por completo todas as coleções de dados, destrói visualmente os controles PictureBox da partida anterior (libertando memória RAM e a área de tela) e reinicia as coordenadas (X e Y). O ciclo do "Exemplo 1" recomeça fluidamente com um novo conjunto de instâncias.

4. Funcionamento do Programa







Form1

Sua Pontuação

21



Pontuação da Banca

22

2♦

5♦

7♠

7♦

10♣

2♠

Q♠

Fim do Turno

i

A Banca estourou! Você venceu

OK

Pedir Carta

Parar

Reiniciar